1.1 Introduction to IoT

Table of Contents

1 Defining IoT

Definition: A dynamic global network infrastructure with self-configuring capabilities based on standard and interoperable communication protocols where physical and virtual "things" have identities, physical attributes, and virtual personalities and use intelligent interfaces, and are seamlessly integrated into the information network, often communicate data associated with users and their environments.

2 Characteristics of IoT

2.1 Dynamic & Self-Adapting:

IoT devices and systems may have the capability to dynamically adapt with the changing contexts and take actions based on their operating conditions, user's context, or sensed environment. For example, consider a surveillance system comprising of a number of surveillance cameras. The surveillance cameras can adapt their modes (to normal or infra-red night modes) based on whether it is day or night. Cameras could switch from lower resolution to higher resolution modes when any motion is detected and alert nearby cameras to do the same. In this example, the surveillance system is adapting itself based on the context and changing (e.g., dynamic) conditions.

2.2 Self-Configuring:

IoT devices may have self-configuring capability, allowing a large number of devices to work together to provide certain functionality (such as weather monitoring). These devices have the ability configure themselves (in association with the IoT infrastructure), setup the networking, and fetch latest software upgrades with minimal manual or user intervention.

2.3 Interoperable Communication Protocols:

IoT devices may support a number of interoperable communication protocols and can communicate with other devices and also with the infrastructure. We describe some of the commonly used communication protocols and models in later sections.

2.4 Unique Identity:

Each IoT device has a unique identity and a unique identifier (such as an IP address or a Uniform Resource Identifier [URI]). IoT systems may have intelligent interfaces which adapt based on the context, allow communicating with users and the environmental contexts. IoT device interfaces allow users to query the devices, monitor their status, and control them remotely, in association with the control, configuration and management infrastructure.

2.5 Integrated into Information Network:

IoT devices are usually integrated into the information network that allows them to communicate and exchange data with other devices and systems. IoT devices can be dynamically discovered in the network, by other devices and/or the network, and have the capability to describe themselves (and their characteristics) to other devices or user applications, For example, a weather monitoring node can describe its monitoring capabilities to another connected node so that they can communicate and exchange data. Integration into the information network helps in making IoT systems “smarter” due to the collective intelligence of the individual devices in collaboration with the infrastructure. Thus, the data from a large number of connected weather monitoring IoT nodes can be aggregated and analysed to predict the weather.

3 Conceptual Framework of IoT

The main tasks of this framework are to analyse and determine the smart activities of these intelligent devices through maintaining a dynamic interconnection among those devices. The proposed framework will help to standardize IoT infrastructure so that it can receive e-services based on context information leaving the current infrastructure unchanged. The active collaboration of these heterogeneous devices and protocols can lead to future ambient computing where the maximum utilization of cloud computing will be ensured.

This model is capable of logical division of physical devices placement, creation of virtual links among different domains, networks and collaborate among multiple application without any central coordination system. IaaS can afford standard functionalities to accommodate and provides access to cloud infrastructure. The service is generally offered by modern data centres maintained by giant companies and organization. It is categorized as virtualization of resources which permits a user to install and run application over virtualization layer and allows the system to be distributed, configurable and scalable.

Total infrastructure system can be categorized into 4 layers to receive context supported e-services out of raw data from the Internet of Things. These 4 layers establish a generic framework that does not alter the current network infrastructure but create an interfacing among services and entities through network virtualization.

3.1 Connectivity Layer-

This layer includes all the physical devices involved in the framework and the interconnection among them. Future internet largely depends on the unification of these common objects found everywhere near us and these should be distinctly identifiable and controllable.

This layer also involves assigning of low range networking devices like sensors, actuators, RFID tags etc and resource management checks the availability of physical resources of all the devices and networks involved in the underlying infrastructure. These devices contain very limited resources and resource management ensures the maximum utilization with little overhead. It also allows sharing and distribution of information among multiple networks or single network divided into multiple domains.

3.2 Access Layer-

Context Data will be reached to internet via IoT Gateway as captured by short range devices in form of raw data. Access layer comprises topology definition, network initiation, creation of domains etc. This layer also includes connection setup, intra-inter domain communication, scheduling, packet transmissions between flow-sensors and IoT gateway. The simulation was run later in this paper for different scenario based on this layer. Feature management contains a feature filter which accepts only acceptable context data and redundant data are rejected. Large number of sensors maintains lots of features but only a small subset of features is useful generate a context data.

Feature filter helps to reduce irrelevant data transmission, increases the data transfer rate of useful data and reduce energy and CPU consumption too. Number of features can be different based on the application requirements and context data types.

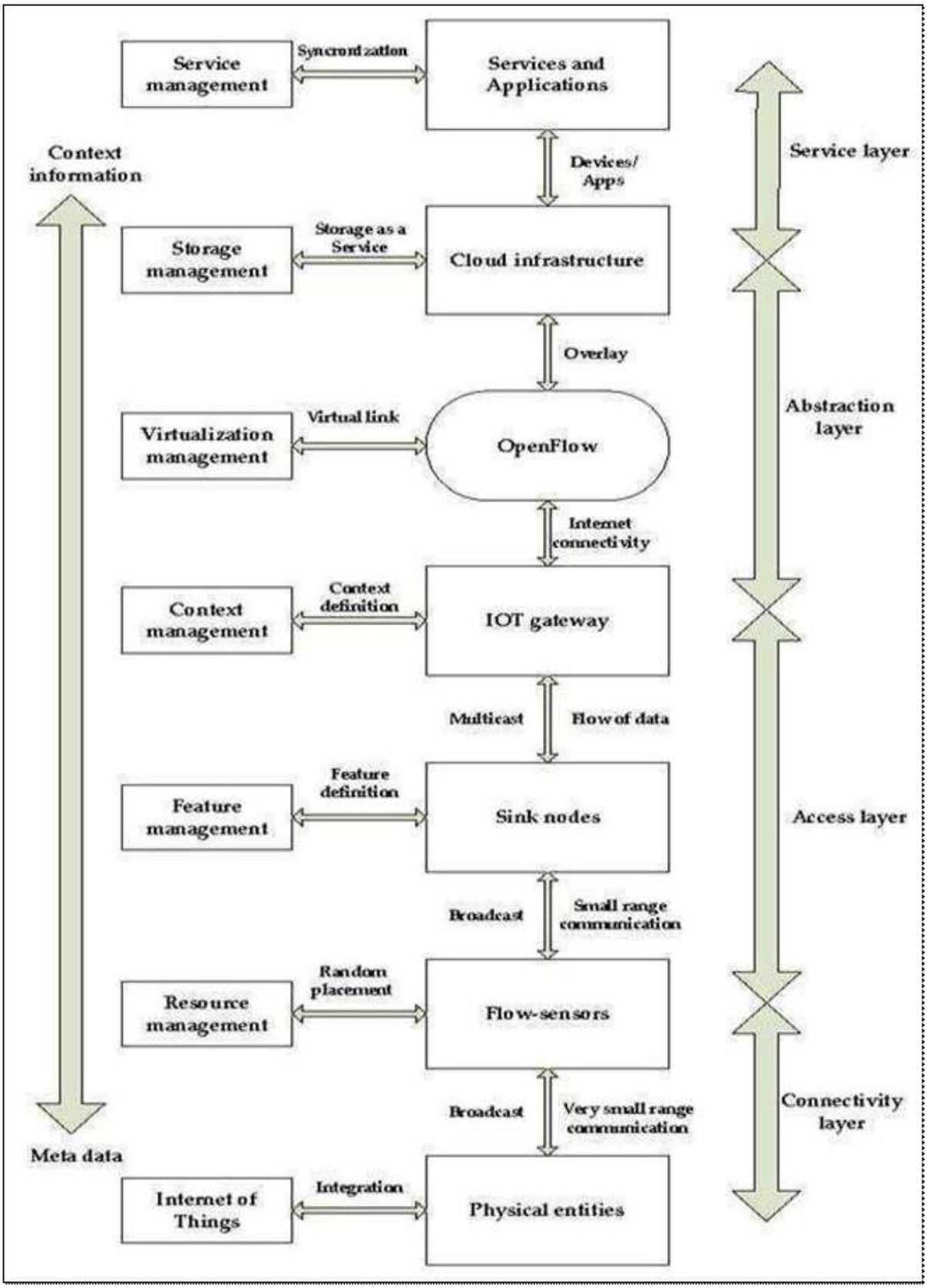

Figure 1: IoT Conceptual View

3.3 Abstraction Layer-

One of the most important characteristics of OpenFlow is to add virtual layers with the preset layers, leaving the established infrastructure unchanged. A virtual link can be created among different networks and a common platform can be developed for various communication systems. The system is fully a centralized system from physical layer viewpoint but a distribution of service (flow visor could be utilized) could be maintained. One central system can monitor, control all sorts of traffics. It can help to achieve better band-width, reliability, robust routing, etc. which will lead to a better Quality of Services (QoS).

In multi-hopping scenario packets are transferred via some adjacent nodes. So, nodes near to access points bears too much load in comparison to distant nodes in a downstream scenario and inactivity of these important nodes may cause the network to be collapsed. Virtual presence of sensor nodes can solve the problem where we can create a virtual link between two sensor networks through access point negotiation. So, we can design a three a three-layer platform, where common platform and virtualization layer are newly added with established infrastructure. Sensors need not to be worried about reach-ability or their placement even in harsh areas. Packet could be sent to any nodes even if it is sited on different networks.

3.4 Service Layer-

Storage management bears the idea about all sorts of unfamiliar and/or important technologies and information which can turn the system scalable and efficient. It is not only responsible for storing data but also to provide security along with it. It also allows accessing data effectively; integrating data to enhance service intelligence, analysis based on the services required and most importantly increases the storage efficiency. Storage and management layer involves data storage & system supervision, software services and business management & operations. Though they are included in one layer, the business support system resides slightly above of cloud computing service whereas Open-Flow is placed below of it as presented to include virtualizations and monitor management.

Service management combines the required services with organizational solutions and thus new generation user service becomes simplified. These forthcoming services are necessitated to be co interrelated and combined in order to meet the demand socio- economic factors such as environment analysis, safety measurement, climate management, agriculture modernization etc.

4 Physical design of IoT

4.1 Things in IoT

The "Things" in IoT usually refers to IoT devices which have unique identities and can perform remote sensing, actuating and monitoring capabilities. IoT devices can exchange data with other connected devices and applications (directly or indirectly), or collect data from other devices and process the data either locally or send the data to centralized servers or cloud-based application back-ends for processing the data, or perform some tasks locally and other tasks within the IoT infrastructure, based on temporal and space constraints (i.e., memory, processing capabilities, communication latencies and speeds, and deadlines).

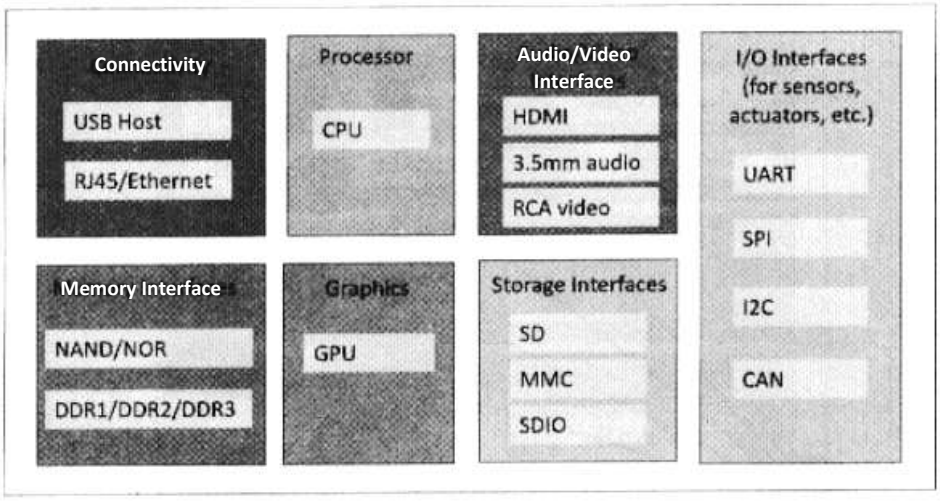

Figure 1.3: Generic block diagram of an IoT Device

Figure 1.3 shows a block diagram of a typical IoT device. An IoT device may consist of several interfaces for connections to other devices, both wired and wireless. These include (i) 1/0 interfaces for sensors, (ii) interfaces for Internet connectivity, (iii) memory and storage interfaces and (iv) audio/video interfaces. An IoT device can collect various types of data from the on-board or attached sensors, such as temperature, humidity, light intensity. The sensed data can be communicated either to other devices or cloud-based servers/storage. IoT devices can be connected to actuators that allow them to interact with other physical entities (including non-IoT devices and systems) in the vicinity of the device. For example, a relay switch connected to an IoT device can turn an appliance on/off based on the commands sent to the IoT device over the Internet.

IoT devices can also be of varied types, for instance, wearable sensors, smart watches, LED lights, automobiles and industrial machines. Almost all IoT devices generate data in some form or the other which when processed by data analytics systems leads to useful information to guide further actions locally or remotely. For instance, sensor data generated by a soil moisture monitoring device in a garden, when processed can help in determining the optimum watering schedules. Figure 1.4 shows different types of IoT devices.

4.2 IoT Protocols

Link Layer Link layer protocols determine how the data is physically sent over the network's physical layer or medium (e.g., copper wire, coaxial cable, or a radio wave). The scope of the link layer is the local network connection to which host is attached. Hosts on the same link exchange data packets over the link layer using link layer protocols. Link layer determines how the packets are coded and signalled by the hardware device over the medium to which the host is attached (such as a coaxial cable). Let us now look at some link layer protocols which are relevant in the context of IoT.

4.2.1 802.3 - Ethernet:

IEEE 802.3 is a collection of wired Ethernet standards for the link layer. For example, 802.3 is the standard for I0BASES Ethernet that uses coaxial cable as a shared medium, 802.3.i is the standard for I0BASE-T Ethernet over copper twisted-pair connections, 802.3] is the standard for 10BASE-F Ethernet over fibre optic connections, 802.3ae is the standard for 10 Gbit/s Ethernet over fibre, and so on. These standards provide data rates from 10 Mb/s to 40 Gb/s and higher. The shared medium in Ethernet can be a coaxial cable, twisted-pair wire or an optical fibre. The shared medium (i.e., broadcast medium) carries the communication for all the devices on the network, thus data sent by

one device can received by all devices subject to propagation conditions and transceiver capabilities. The specifications of the 802.3 standards are available on the 802.3 working group website [2].

4.2.2 802.11 - WiFi:

IEEE 802.11 is a collection of wireless local area network (WLAN) communication standards, including extensive description of the link layer. For example, 802.11a operates in the 5 GHz band, 802.11b and 802.11g operate in the 2.4 GHz band, 802.11n operates in the 2.4/5 GHz bands, 802.11ac operates in the 5 GHz band and 802.11ad operates in the 60 GHz band. These standards provide data rates from 1 Mb/s to upto 6.75 Gb/s. The specifications of the 802.11 standards are available on the IEEE 802.11 working group website [3]

4.2.3 802.16 - WiMax:

IEEE 802.16 is a collection of wireless broadband standards, including extensive descriptions for the link layer (also called WiMax). WiMax standards provide data rates from 1.5 Mb/s to 1 Gb/s. The recent update (802.16m) provides data rates of 100 Mbit/s for mobile stations and 1 Gbit/s for fixed stations. The specifications of the 802.11 standards are readily available on the IEEE 802.16 working group website [4]

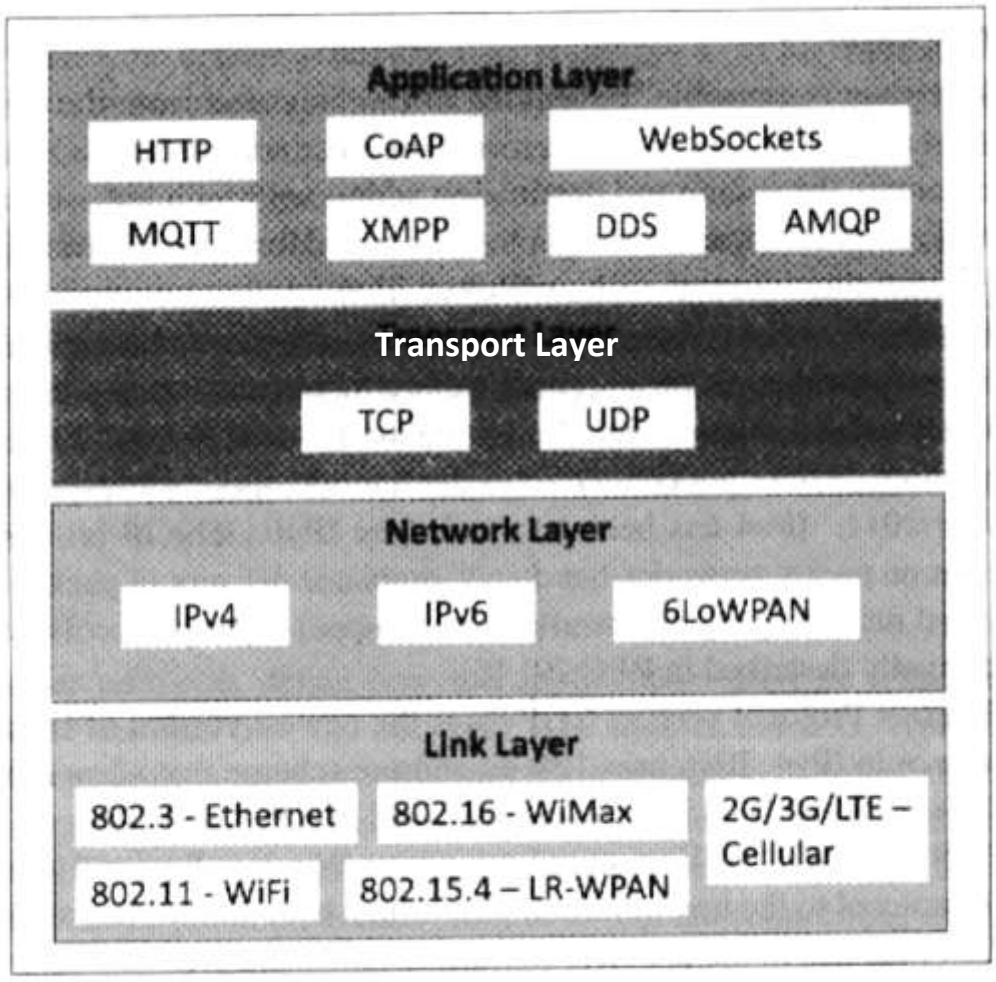

Figure 1.5: IoT Protocols

4.2.4 802.154 - LR-WPAN:

IEEE 802,15.4 is a collection of standards for low-rate wireless personal area networks (LR-WPANS). These standards form the basis of specifications for high level communication protocols such as ZigBee. LR-WPAN standards provide data rates from 40 Kb/s 250 Kb/s. These standards provide low-cost and low-speed communication for power constrained devices. The specifications of the 802.15.4 standards are available on the IEEE, 802,15 working group websites [5]

4.2.5 2G/ 3G/ 4G - Mobile Communication:

There are different generations of mobile communication standards including second generation (2G including GSM and CDMA), third generation (3G - including UMTS and CDMA2000) and fourth generation (4G - including LTE). IoT devices based on these standards can communicate over cellular networks, Data rates for these standards range from 9.6 Kb/s (for 2G) to up to 100 Mb/s (for 4G) and are available from the 3GPP websites.

4.3 Network/Internet Layer

The network layers are responsible for sending of IP datagrams from the source network to the destination network. This layer performs the host addressing and packet routing, The datagrams contain the source and destination addresses which are used to route them from the source to destination across multiple networks. Host identification is done using hierarchical IP addressing schemes such as IPv4 or IPv6.

4.3.1 IPv4:

Internet Protocol version 4 (IPv4) is the most deployed Internet protocol that is used to identify the devices on a network using a hierarchical addressing scheme. IPv4 uses a 32-bit address scheme that allows total of 2%? or 4,294,967,296 addresses. As more and more devices got connected to the Internet, these addresses got exhausted in the year 2011. IPv4 has been succeeded by IPv6. The IP protocols establish connections on packet networks, but do not guarantee delivery of packets. Guaranteed delivery and data integrity are handled by the upper layer protocols (such as TCP). [Pv4 is formally described in RFC 791 [6].

4.3.2 IPv6:

Internet Protocol version 6 (IPv6) is the newest version of Internet protocol and successor to [Pv4, IPv6 uses 128-bit address scheme that allows total of 2'2% or 3.4 x 10°® addresses. IPv4 is formally described in RFC 2460 [7].

4.3.3 6LoWPAN:

6LoWPAN (IPv6 over Low power Wireless Personal Area Networks) brings IP protocol to the low-power devices which have limited processing capability. 6LoWPAN operates in the 2.4 GHz frequency range and provides data transfer rates of 250 Kb/s. 6LoWPAN works with the 802.15.4 link layer protocol and defines compression mechanisms for [Pv6 datagrams over IEEE 802.15.4-based networks [8].

4.4 Transport Layer

The transport layer protocols provide end-to-end message transfer capability independent of the underlying network. The message transfer capability can be set up on connections, either using handshakes (as in TCP) or without handshakes/acknowledgements (as in UDP). The transport layer provides functions such as error control, segmentation, flow control and congestion control.

4.4.1 TCP:

Transmission Control Protocol | (TCP) is the most widely used transport layer protocol, that is used by web browsers (along with HTTP, HTTPS application layer protocols), email programs (SMTP application layer protocol) and file transfer (FTP). TCP is a connection oriented and stateful protocol. While IP protocol deals with sending packets, TCP ensures reliable transmission of packets in-order. TCP also provides error detection capability so that duplicate packets can be discarded and lost packets are retransmitted. The flow control capability of TCP ensures that rate at which the sender sends the data is not too high for the receiver to process. The congestion control capability of TCP helps in avoiding network congestion and congestion collapse which can lead to degradation of network performance. TCP is described in RFC 793 [9].

4.4.2 UDP:

Unlike TCP, which requires carrying out an initial setup procedure, UDP is a connectionless protocol. UDP is useful for time-sensitive applications that have very small data units to exchange and do not want the overhead of connection setup. UDP is a transaction oriented and stateless protocol. UDP does not provide guaranteed delivery, ordering of messages and duplicate elimination. Higher levels of protocols can ensure reliable delivery or ensuring connections created are reliable. UDP is described in RFC 768 [10].

4.5 Application Layer

Application layer protocols define how the applications interface with the lower layer protocols to send the data over the network. The application data, typically in files, is encoded by the application layer protocol and encapsulated in the transport layer protocol which provides connection or transaction-oriented communication over the network. Port numbers are used for application addressing (for example port 80 for HTTP, port 22 for SSH, etc.). Application layer protocols enable process-to-process connections using ports.

4.5.1 HTTP:

Hypertext Transfer Protocol (HTTP) is the application layer protocol that forms the foundation of the World Wide Web (WWW). HTTP includes commands such as GET, PUT, POST, DELETE, HEAD, TRACE, OPTIONS, etc. The protocol follows a request-response model where a client sends requests to a server using the HTTP commands. HTTP is a stateless protocol and each HTTP request is independent of the other requests. An HTTP client can be a browser or an application running on the client (e.g., an application running on an IoT device, a mobile application or other software). HTTP protocol uses Universal Resource Identifiers (URIs) to identify HTTP resources. HTTP is described in RFC 2616 [11].

4.5.2 CoAP:

Constrained Application Protocol (CoAP) is an application layer protocol for machine-to-machine (M2M) applications, meant for constrained environments with constrained devices and constrained networks. Like HTTP, CoAP is a web transfer protocol and uses a request-response model, however it runs on top of UDP instead of TCP. CoAP uses a client-server architecture where clients communicate with servers using connectionless datagrams. CoAP is designed to easily interface with HTTP. Like HTTP, CoAP supports methods such as GET, PUT, POST, and DELETE. CoAP draft specifications are available on IEFT Constrained environments (CoRE) Working Group website [12].

4.5.3 WebSocket:

WebSocket protocol allows full-duplex communication over a single socket connection for sending messages between client and server. WebSocket is based on TCP and allows streams of messages to

be sent back and forth between the client and server while keeping the TCP connection open. The client can be a browser, a mobile application or an IoT device. WebSocket is described in RFC 6455 [13].

4.5.4 MQTT:

Message Queue Telemetry Transport (MQTT) is a light-weight messaging protocol based on the publish-subscribe model. MQTT uses a client-server architecture where the client (such as an IoT device) connects to the server (also called MQTT Broker) and publishes messages to topics on the server. The broker forwards the messages to the clients subscribed to topics. MQTT is well suited for constrained environments where the devices have limited processing and memory resources and the network bandwidth is low. MQTT specifications are available on IBM developer Works (14).

4.5.5 XMPP:

Extensible Messaging and Presence Protocol (XMPP) is a protocol for real-time communication and streaming XML data between network entities. XMPP powers wide range of applications including messaging, presence, data syndication, gaming, multi-party chat and voice/video calls. XMPP allows sending small chunks of XML data from one network entity to another in near real-time. XMPP is a decentralized protocol and uses a client-server architecture, XMPP supports both client-to-server and server-to-server communication paths. In the context of IoT, XMPP allows real-time communication between IoT devices, XMPP is described in RFC 6120 [15].

4.5.6 DDS:

Data Distribution Service (DDS) is a data-centric middleware standard for device-to-device or machine-to-machine communication. DDS uses a publish-subscribe model where publishers (e.g., devices that generate data) create topics to which subscribers (e.g., devices that want to consume data) can subscribe. Publisher is an object responsible for data distribution and the subscriber is responsible for receiving published data. DDS provides quality-of-service (QoS) control and configurable reliability. DDS is described in Object Management Group (OMG) DDS specification [16).

4.5.7 AMQP:

Advanced Message Queuing Protocol (AMQP) is an open application layer protocol for business messaging. AMQP supports both point-to-point and publisher/subscriber models, routing and queuing. AMQP brokers receive messages from publishers (e.g., devices or applications that generate data) and route them over connections to consumers (applications that process data). Publishers publish the messages to exchanges which then distribute message copies to queues. Messages are cither delivered by the broker to the consumers which have subscribed to the queues or the consumers can pull the messages from the queues. AMQP specification is available on the AMQP working group website [17].

5 Logical Design of IoT

Logical design of an IoT system refers to an abstract representation of the entities and processes without going into the low-level specifics of the implementation. In this section we describe the functional blocks of an IoT system and the communication APIs that are used for the examples in these notes.

5.1 IoT Functional Blocks

An IoT system comprises of a number of functional blocks that provide the system the capabilities for identification, sensing, actuation, communication, and management as shown in Figure 1.6. These functional blocks are described as follows:

- **Device**: An IoT system comprises of devices that provide sensing, actuation, monitoring and control functions.
- **Communication**: The communication block handles the communication for the IoT system.
- **Services**: An IoT system uses various types of IoT services such as services for device monitoring, device control services, data publishing services and services for device discovery.
- **Management**: Management functional block provides various functions to govern the IoT system.
- **Security**: Security functional block secures the IoT system and by providing functions such as authentication, authorization, message and content integrity, and data security.
- **Application**: IoT applications provide an interface that the users can use to control and monitor various aspects of the IoT system. Applications also allow users to view the system status and view or analyse the processed data.

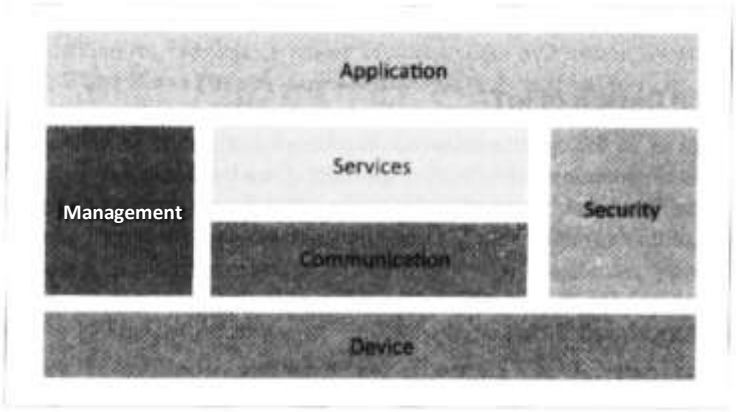

Figure 1.6: Functional Blocks of IoT

5.2 IoT Communication Models

5.2.1 Request-Response:

Request-Response is a communication model in which the client sends requests to the server and the server responds to the requests. When the server receives a request, it decides how to respond, fetches the data, retrieves resource representations, prepares the response, and then sends the response to the client. Request-Response model is a stateless communication model and each request-response pair is independent of others. Figure 1.7 shows the client-server interactions in the request-response model.

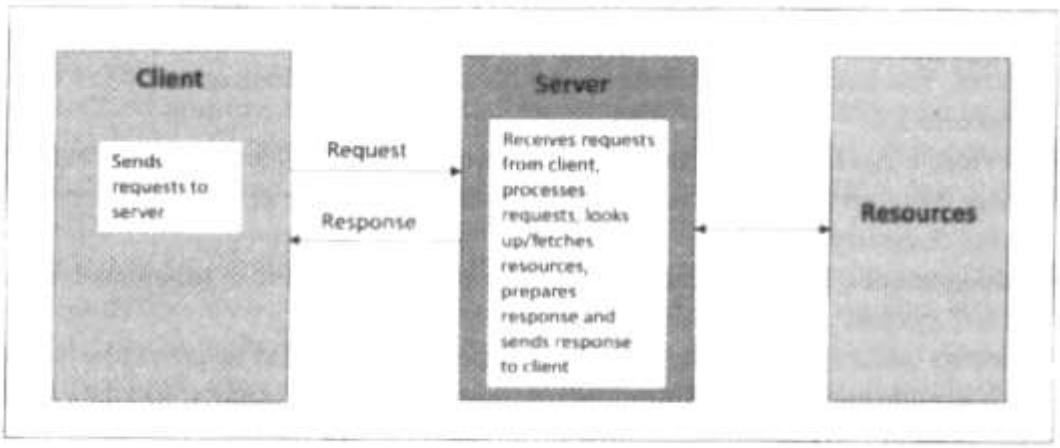

Figure 1.7: Request-Response communication model

5.2.2 Publish-Subscribe:

Publish-Subscribe is a communication model that involves publishers, brokers and consumers. Publishers are the source of data. Publishers send the data to the topics which are managed by the broker. Publishers are not aware of the consumers. Consumers subscribe to the topics which are managed by the broker. When the broker receives data for a topic from the publisher, it sends the data to all the subscribed consumers. Figure 1.8 shows the publisher-broker-consumer interactions in the publish-subscribe model.

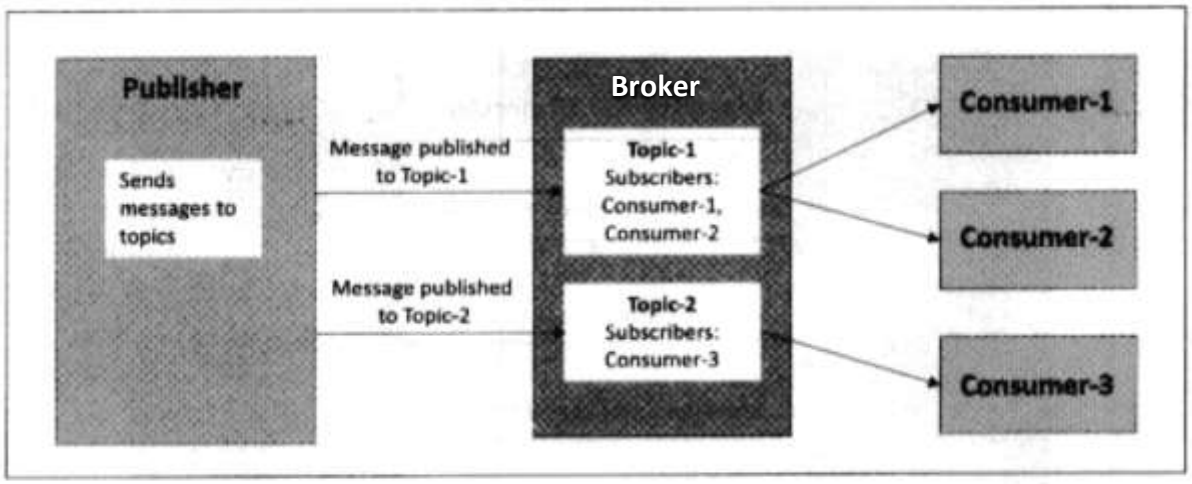

Figure 1.8: Publish-Subscribe communication model

5.2.3 Push-Pull:

Push-Pull is a communication model in which the data producers push the data to queues and the consumers pull the data from the queues. Producers do not need to be aware of the consumers. Queues help in decoupling the messaging between the producers and consumers. Queues also act as a buffer which helps in situations when there is a mismatch between the rate at which the producers push data and the rate at which the consumers pull data. Figure 1.9 shows the publisher-queue-consumer interactions in the push-pull model.

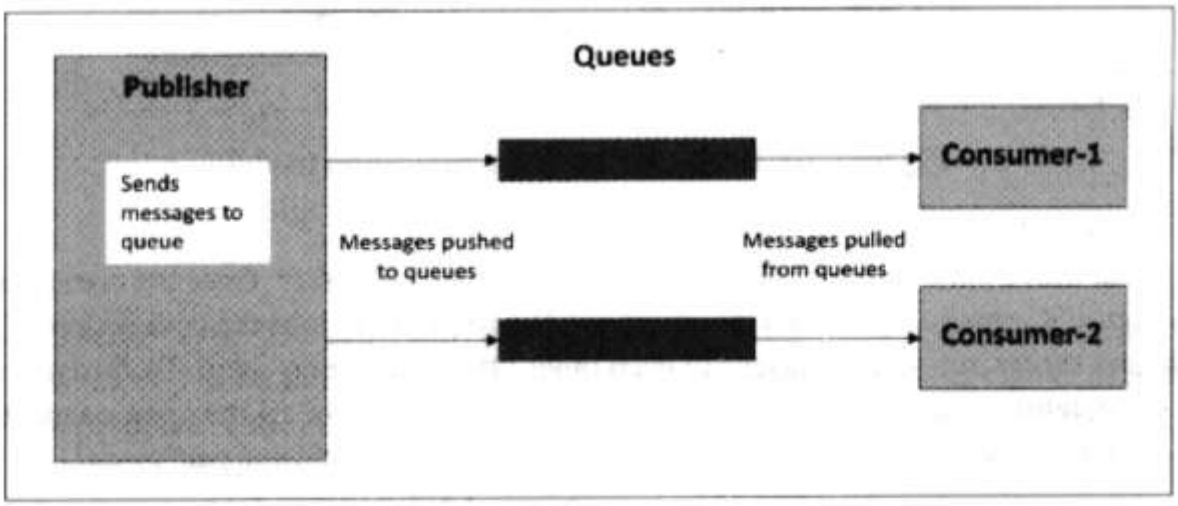

Figure 1.9: Push-Pull communication model

5.2.4 Exclusive Pair:

Exclusive Pair is a bi-directional, fully duplex communication model that uses a persistent connection between the client and server. Once the connection is setup it remains open until the client sends a request to close the connection. Client and server can send messages to each other after connection setup. Exclusive pair is a stateful communication model and the server is aware of all the open connections. Figure 1.10 shows the client-server interactions in the exclusive pair model.

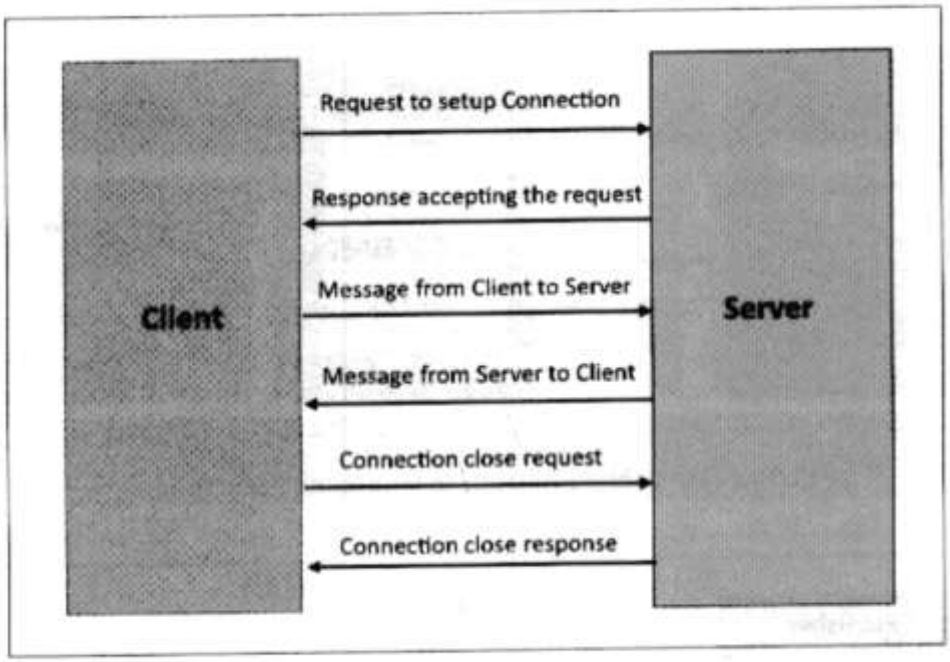

Figure 1.10: Exclusive Pair communication model

5.3 IoT Communication APIs

In the previous section you learned about various communication models. In this section you will learn about two specific communication APIs which are used in the examples in this book.

5.3.1 REST-based Communication APIs

Representational State Transfer (REST) [88] is a set of architectural principles by which you can design web services and web APIs that focus on a system's resources and how resource states are addressed and transferred. REST APIs follow the request-response apply to the components, connectors, and data elements, within a distributed hypermedia system. The REST **architectural constraints** are as follows:

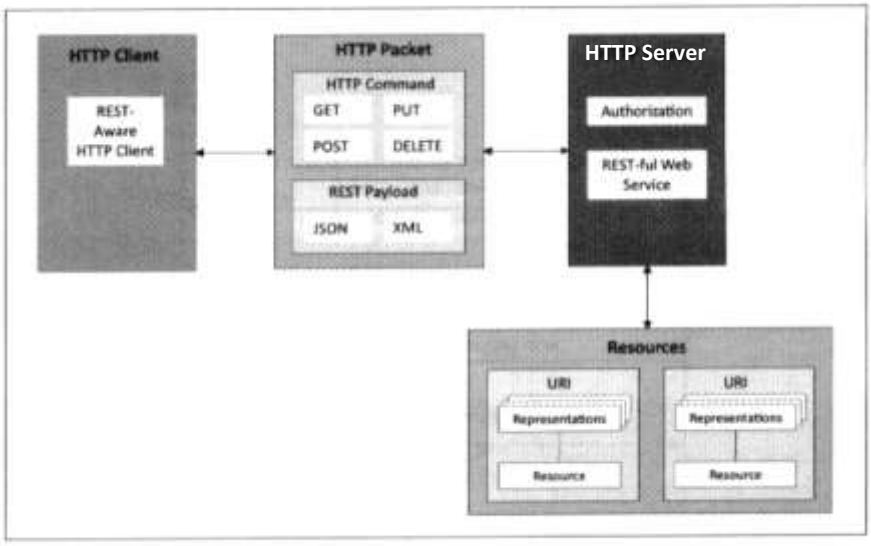

Figure 1.11: Communication with REST APIs

- **Client-Server:** The principle behind the client-server constraint is the separation of concerns. For example, clients should not be concerned with the storage of data which is a concern of the server. Similarly, the server should not be concerned about the user interface, which is a concern of the client. Separation allows client and server to be independently developed and updated.
- **Stateless:** Each request from client to server must contain all the information necessary to understand the request, and cannot take advantage of any stored context on the server. The session state is kept entirely on the client.
- **Cache-able:** Cache constraint requires that the data within a response to a request be implicitly or explicitly labelled as cache-able or non-cache-able. If a response is cache-able, then a client cache is given the right to reuse that response data for later, equivalent requests. Caching can partially or completely eliminate some interactions and improve efficiency and scalability.
- **Layered System:** Layered system constraint, constrains the behaviour of components such that each component cannot see beyond the immediate layer with which they are interacting. For example, a client cannot tell whether it is connected directly to the end server, or to an intermediary along the way. System scalability can be improved by allowing intermediaries to respond to requests instead of the end server, without the client having to do anything different.
- **Uniform Interface:** Uniform Interface constraint requires that the method of communication between a client and a server must be uniform. Resources are identified in the requests (by URIs in web-based systems) and are themselves separate from the representations of the resources that are returned to the client. When a client holds a representation of a resource it has all the information required to update or delete the resource (provided the client has

required permissions). Each message includes enough information to describe how to process the message.

- **Code on demand:** Servers can provide executable code or scripts for clients to execute in their context. This constraint is the only one that is optional.

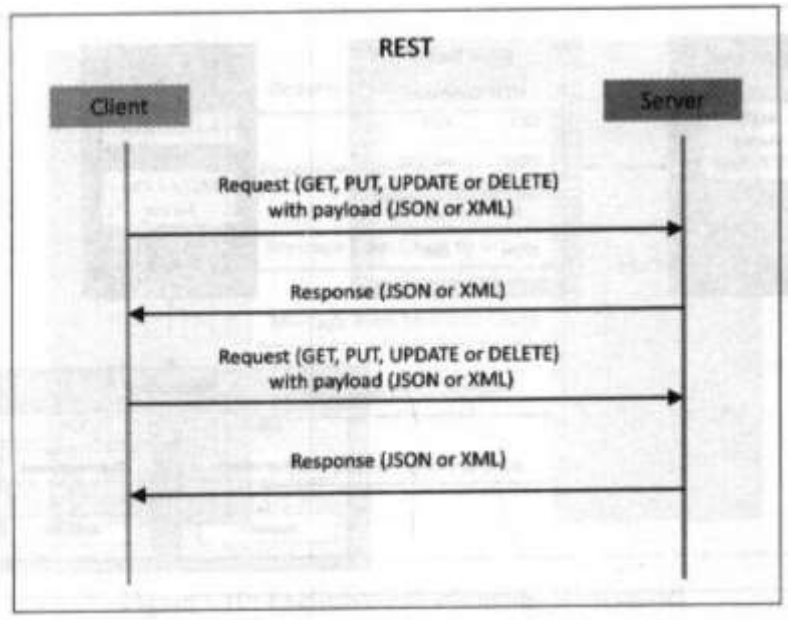

Figure 1.12: Request-response model used by REST

A RESTful web service is a "web API" implemented using HTTP and REST principles. Figure 1.11 shows the communication between client and server using REST APIs. Figure 1.12 shows the interactions in the request-response model used by REST. RESTful web service is a collection of resources which are represented by URIs. RESTful web API has a base URI (e.g., http://example.com/api/tasks/). The clients send requests to these URIs using the methods defined by the HTTP protocol (e.g., GET, PUT, POST, or DELETE), as shown in Table 1.1.

HTTP Method	Resource Type	Action	Example
GET	Collection URI	List all the resources in a collection	http://example.com/api/tasks/ (list all tasks)
GET	Element URI	Get information about a resource	http://example.com/api/tasks/1/ (get information on task-1)
POST	Collection URI	Create a new resource	http://example.com/api/tasks/ (create a new task from data provided in the request)
POST	Element URI	Generally not used	
PUT	Collection URI	Replace the entire collection with another collection	http://example.com/api/tasks/ (replace entire collection with data provided in the request)
PUT	Element URI	Update a resource	http://example.com/api/tasks/1/ (update task-1 with data provided in the request)
DELETE	Collection URI	Delete the entire collection	http://example.com/api/tasks/ (delete all tasks)
DELETE	Element URI	Delete a resource	http://example.com/api/tasks/1/ (delete task-1)

Table 1.1: HTTP request methods and actions

A RESTful web service can support various Internet media types (JSON being the most popular media type for RESTful web services). IP for Smart Objects Alliance (IPSO Alliance) has published an Application Framework that defines a RESTful design for use in IP smart object systems [18].

5.3.2 WebSocket-based Communication APIs

WebSocket APIs allow bi-directional, full duplex communication between clients and servers, WebSocket APIs follow the exclusive pair communication model described in previous section and as shown in Figure 1.13. Unlike request-response APIs such as REST, the

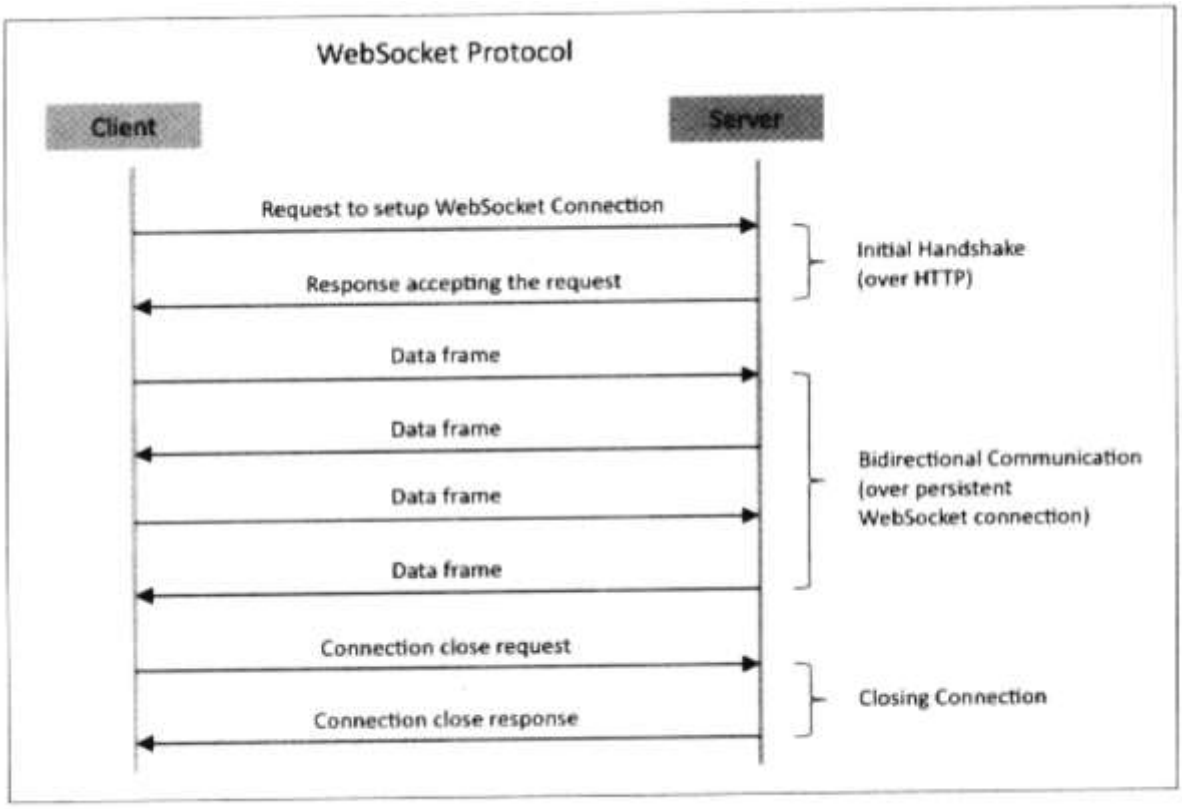

Figure 1.13: Exclusive pair model used by WebSocket APIs

WebSocket APIs allow full duplex communication and do not require a new connection to be setup for each message to be sent. WebSocket communication begins with a connection setup request sent by the client to the server. This request (called a WebSocket handshake) is sent over HTTP and the server interprets it as an upgrade request. If the server supports WebSocket protocol, the server responds to the WebSocket handshake response. After the connection is setup, the client and server can send data/messages to each other in full-duplex mode. WebSocket APIs reduce the network traffic and latency as there is no overhead for connection setup and termination requests for each message. WebSocket is suitable for IoT applications that have low latency or high throughput requirements.